Báo cáo Lab 0 - System Call: `trace`

Thông tin nhóm:

STT	MSSV	Họ và tên
1	24120010	Trần Văn Tấn Khôi
2	24120035	Lê Thành Đạt
3	24120361	Trịnh Tuấn Kiệt

1. Tóm tắt về quá trình thực hiện

Để hoàn thành bài lab, nhóm tìm hiểu cơ chế chuyển đổi từ user-space sang kernel-space thông qua lệnh `ecall` của RISC-V. Quá trình cài đặt bao gồm một số bước đáng chú ý gồm:

- Lưu trạng thái trace: Thêm trường `trace_mask` trong struct `proc` trong `kernel/proc.h` để lưu trữ bitmask của từng tiến trình.
- Định tuyến hệ thống: Định nghĩa system call mới (`SYS_trace`), cấu hình stub trong không gian người dùng (`user/usys.pl`, `user/user.h`) và ánh xạ hàm xử lý `sys_trace` trong `kernel/syscall.c`.
- Đăng ký chương trình người dùng: thêm `$U/_trace` vào danh sách `UPROGS` trong `Makefile` để biên dịch chương trình `trace` và đưa vào `fs.img`.
- Ghi lại log trace: Tại hàm `syscall()`, kiểm tra số hiệu của system call có khớp với bit trong `trace_mask` hay không để in ra thông tin kèm giá trị trả về.

2. Một số vấn đề gặp phải

2.1. Lỗi tương thích trình biên dịch hiện tại (GCC 14/15) và kiến trúc QEMU

Khi biên dịch mã nguồn gốc của xv6 trên môi trường hiện đại, quá trình build gặp lỗi từ chối biên dịch (`-Werror`) với hai nguyên nhân chính:

- Lỗi kiểu con trỏ hàm (`rwsbrk`): GCC 14+ đẩy `-Wincompatible-pointer-types` thành lỗi cứng. Hàm `rwsbrk()` khai báo danh sách tham số trống `void(void)` không khớp với kiểu con trỏ hàm `void(*)(char*)` sử dụng trong bảng `quicktests[]`.
- Lỗi tập lệnh phần cứng (GCC 15/QEMU 10): `Makefile` không chỉ rõ `-march`, khiến trình biên dịch chéo tự động bật phần mở rộng vector. Điều này khiến hàm `memcpy` trong `fork()` bị tự động vector hóa thành các lệnh `vsetvtil`/`vle8.v` và `entry.S` phát sinh lệnh `c.mul`, dẫn đến bẫy lệnh bất hợp pháp (`-illegal-instruction-traps`) làm hệ thống bị treo trong im lặng trước khi hiển thị đầu ra trên console.

Giải pháp khắc phục:

- Sửa lại khai báo của hàm `rwsbrk` trong mã nguồn `usertests.c` với tham số `char*` để khớp kiểu con trỏ hàm của bảng kiểm thử.
- Cấu hình lại `Makefile` để ghim tường minh cờ `-march=rv64g` cho cả `CFLAGS` và `ASFLAGS`, qua đó loại bỏ các lệnh vector không tương thích với CPU mặc định của QEMU 10.

2.2. Quản lý vòng đời của `trace_mask` (`fork()` và `allocproc()`)

Vì `trace_mask` được lưu trong `struct proc`, giá trị này cần được quản lý đúng qua vòng đời của tiến trình. Ban đầu chỉ gán `trace_mask` cho tiến trình gọi system call `trace()` mà chưa tính tới hai trường hợp sau:

- Tiến trình mới được sinh ra bởi `fork()`: cần sao chép bitmask từ tiến trình cha sang tiến trình con (`np->trace_mask = p->trace_mask;`) trong `kernel/proc.c`. Nếu thiếu bước này, các tiến trình con sẽ không kế thừa trạng thái trace của tiến trình cha.
- Tiến trình được tái sử dụng từ bảng `proc`: hàm `allocproc()` không khởi tạo zero toàn bộ cấu trúc, nên cần gán rõ `p->trace_mask = 0;` để tránh tiến trình mới thừa hưởng giá trị cũ của tiến trình đã kết thúc.

Lưu ý: trong chương trình `user/trace.c` được cung cấp sẵn, tiến trình gọi `trace()` sẽ thực hiện `exec()` chứ không phải `fork()`, nên hai trường hợp trên không xuất hiện trong các lệnh kiểm thử ở mục 3. Phần xử lý này được bổ sung nhằm bảo đảm hành vi đúng khi một tiến trình đã bật trace thực sự tạo ra tiến trình con.

2.3. Đồng bộ hóa danh sách tên System Call (`syscall_names`)

Khi in log, kernel cần ánh xạ chỉ số của system call sang tên tương ứng. Nếu khai báo mảng `syscall_names[]` theo thứ tự thủ công, rất dễ xảy ra lệch chỉ số khi `kernel/syscall.h` được bổ sung system call mới, dẫn đến in sai tên hoặc gặp con trỏ NULL.

Hướng xử lý: khai báo mảng bằng designated initializer gắn trực tiếp với từng macro `SYS_*`, ví dụ `[SYS_trace] "trace"`. Cách này bảo đảm tên luôn bám theo định nghĩa trong `kernel/syscall.h`, không phụ thuộc vào thứ tự khai báo nên không thể lệch chỉ số.

Về phần an toàn chỉ số: `num` đã được kiểm tra bởi điều kiện điều phối sẵn có trong hàm `syscall()` (`num > 0 && num < NELEM(syscalls) && syscalls[num]`), nên bước in log chỉ được thực hiện khi `num` chắc chắn nằm trong phạm vi hợp lệ. Vì vậy không cần thêm điều kiện bảo vệ riêng cho bước in log.

3. Kiểm thử và Xác thực

Quá trình cài đặt được kiểm chứng trên môi trường QEMU. Kết quả thử nghiệm trả về đúng các syscall được lọc và các giá trị tương ứng:

```
$ trace 32 grep hello README
syscall read -> 1023
syscall read -> 971
syscall read -> 298
syscall read -> 0
```

```
$ trace 2147483647 grep hello README
syscall trace -> 0
syscall exec -> 3
syscall open -> 3
syscall read -> 1023
syscall read -> 971
syscall read -> 298
syscall read -> 0
syscall close -> 0
```

Lưu ý: giá trị số byte mà `read` trả về phụ thuộc vào kích thước tệp `README` trong filesystem của xv6, nên con số khác với ví dụ trong đề bài. Cấu trúc, thứ tự và định dạng của từng dòng log là tương ứng.

Để kiểm chứng yêu cầu "`trace` không ảnh hưởng tới các tiến trình khác", ta chạy `grep` mà không bật trace:

```
$ grep hello README
$
```

Không xuất hiện bất kỳ dòng log nào. Về mặt kỹ thuật, `trace_mask` nằm trong `struct proc` và chỉ được thay đổi qua system call `trace()`, nên các tiến trình khác trong hệ thống vẫn giữ `trace_mask = 0` và không in gì.